

Evaluation of a set of functions related to Ripley's K analysis

Gabriel Arellano

2026-05-21

Goal

Evaluate, understand, and decide over a set of related functions that count individuals in circular areas and compare those counts with a null expectation, implementing Ripley's K test. At the moment, this is an isolated set of functions; see "1-dependencies-and-priorities-2026-05-20.pdf" report.

This set contains the following functions:

- "CalcRingArea" in `spatial > RipUvK.r`
- "partialcirclearea" in `spatial > RipUvK.r`
- "circlearea" in `spatial > RipUvK.r`
- "Annuli" in `spatial > RipUvK.r`
- "NDcount" in `spatial > NeighborDensityFun.r`
- "NeighborDensities" in `spatial > NeighborDensityFun.r`
- "RipUvK" in `spatial > RipUvK.r`

Overall criteria

These functions are located in two different scripts but all are related to the same type of calculation: counting number of neighbors in a given area, and compare that with some null expectation (Ripley's K function). The set covers all code in `RipUvK.r` and `NeighborDensityFun.r` original scripts.

This type of calculation is widely used in ForestGEO / ATFS, for example to estimate competition around target individuals and so on. Perhaps not to much for point pattern analyses. But,

in general, counting points in certain areas around target individuals (or locations) is valuable. Perhaps most of these functions can be incorporated into a more general function that counts (or summarizes) data belonging to different classes, within areas that have a minimum and a maximum radius.

Modelling or comparison with null expectations is less valuable for the fgeo2 package. The Ripley's K function is surely maintained by other packages: spatstat::Kest and spatstat::Lest may be the gold standard (and fastest?). The code implements a well-known mathematical formula so it should not require a lot of maintenance. However, the RipUvK function is a clear candidate to remove, and substitute by a more general tutorial on how to apply point pattern analysis to forest data.

Preliminaries for code evaluation

First, we declare the path to fgeo2 folder and all the functionality needed, including CTFS / fgeo functions, external packages, and auxiliary code:

```
# This is my local path. You may have to adapt this.
g <- regexpr("fgeo2", getwd())
path <- substr(getwd(), 1, g + attr(g, "match.length"))

# Source useful functionality:
source(paste0(path, "evaluation/0-code-to-source.r"))

# Other functionality:
library(igraph)
```

Warning: package 'igraph' was built under R version 4.5.2

Attaching package: 'igraph'

The following objects are masked from 'package:stats':

decompose, spectrum

The following object is masked from 'package:base':

union

```
library(fgeo)
```

```
-- Attaching packages ----- fgeo 1.1.4 --
```

```
v fgeo.analyze 1.1.15      v fgeo.tool    1.2.10
v fgeo.plot    1.1.11      v fgeo.x       1.1.4
```

```
-- Conflicts ----- fgeo_conflicts() --
x fgeo.tool::filter() masks stats::filter()
```

```
# This chunk of code will source all the scripts:
f1 <- list.files(paste0(path, "01_utilities/R/new/"), full.names = TRUE)
f2 <- list.files(paste0(path, "02_biomass/R/new/"), full.names = TRUE)
f3 <- list.files(paste0(path, "03_spatial/R/new/"), full.names = TRUE)
f4 <- list.files(paste0(path, "04_abundance_diversity/R/new/"), full.names = TRUE)
f5 <- list.files(paste0(path, "05_demography_growth/R/new/"), full.names = TRUE)
f6 <- list.files(paste0(path, "06_topography_habitat/R/new/"), full.names = TRUE)
f7 <- list.files(paste0(path, "07_mapping_plotting/R/new/"), full.names = TRUE)
ff <- c(f1, f2, f3, f4, f5, f6, f7)
for(f in ff) source(f)
```

Current source code

```
# Use this to see the source code, manually.
```

```
CalcRingArea
```

```
function (data, radius, plotdim)
{
  nopts <- dim(data)[1]
  internalArea <- numeric()
  for (i in 1:nopts) {
    xdists <- data$gx[i]
    if (plotdim[1] - data$gx[i] < xdists)
      xdists <- plotdim[1] - data$gx[i]
    internalArea[i] <- partialcirclearea(radius, xdists, data$gy[i],
      plotdim[2] - data$gy[i])
  }
  return(list(total = sum(internalArea), each = internalArea))
}
```

partialcirclearea

```
function (r, c2, cy1, cy2)
{
  if (cy1 <= cy2) {
    c1 <- cy1
    c3 <- cy2
  }
  else {
    c1 <- cy2
    c3 <- cy1
  }
  if (r > c1) {
    alpha1 <- acos(c1/r)
    y1 <- sqrt(r * r - c1 * c1)
  }
  if (r > c2) {
    alpha2 <- acos(c2/r)
    y2 <- sqrt(r * r - c2 * c2)
  }
  if (r > c3) {
    alpha3 <- acos(c3/r)
    y3 <- sqrt(r * r - c3 * c3)
  }
  cornerdist1 <- sqrt(c1 * c1 + c2 * c2)
  cornerdist3 <- sqrt(c2 * c2 + c3 * c3)
  if (r <= c1 && r <= c2)
    area <- circlearea(r)
  else if (r > c1 && r <= c2 && r <= c3)
    area <- r * r * (pi - alpha1) + c1 * y1
  else if (r <= c1 && r > c2 && r <= c3)
    area <- r * r * (pi - alpha2) + c2 * y2
  else if (r > c1 && r > c2 && r <= c3) {
    if (r > cornerdist1)
      area <- r * r * (3 * pi/4 - alpha1/2 - alpha2/2) +
        c1 * c2 + (c1 * y1 + c2 * y2)/2
    else area <- r * r * (pi - alpha1 - alpha2) + c1 * y1 +
      c2 * y2
  }
  else if (r <= c2 && r > c1 && r > c3)
    area <- r * r * (pi - alpha1 - alpha3) + c1 * y1 + c3 *
      y3
}
```

```

else if (r > c2 && r > c1 && r > c3) {
  if (r > cornerdist3)
    area <- r * r * (pi - alpha1 - alpha3)/2 + c1 * c2 +
      c2 * c3 + (c1 * y1 + c3 * y3)/2
  else if (r > cornerdist1 && r <= cornerdist3)
    area <- r * r * (3 * pi/4 - alpha1/2 - alpha2/2 -
      alpha3) + (c1 * y1 + c2 * y2)/2 + c3 * y3 + c1 *
      c2
  else if (r <= cornerdist1 && r <= cornerdist3)
    area <- r * r * (pi - alpha1 - alpha2 - alpha3) +
      c1 * y1 + c2 * y2 + c3 * y3
}
return(area)
}

```

circlearea

```

function (r)
{
  return(pi * r^2)
}

```

Annuli

```

function (spdata, r, plotdim)
{
  TotalAreaPerRing <- numeric()
  AreaPerCircle <- matrix(nrow = dim(spdata)[1], ncol = length(r) +
    1)
  TotalAreaPerRing[1] <- 0
  AreaPerCircle[, 1] <- 0
  for (i in 1:length(r)) {
    ringarea <- CalcRingArea(spdata, r[i], plotdim)
    TotalAreaPerRing[i + 1] <- ringarea$total
    AreaPerCircle[, i + 1] <- ringarea$each
  }
  AreaPerRing <- t(apply(AreaPerCircle, 1, diff))
  return(list(total = diff(TotalAreaPerRing), each = AreaPerRing))
}

```

NDcount

```
function (censdata, r = 20, plotdim = c(1000, 500))
{
  sppi.pts <- with(censdata, data.frame(gx, gy))
  areas <- CalcRingArea(sppi.pts, r, plotdim)$each
  poly <- spoints(c(0, 0, plotdim[1], 0, plotdim[1], plotdim[2],
    0, plotdim[2]))
  counts <- khat(sppi.pts, poly = poly, r, newstyle = TRUE)$counts
  output <- counts * pi * r^2/areas
  return(as.vector(output))
}
```

NeighborDensities

```
function (censdata, censdata2 = NULL, r = 20, plotdim = c(1000,
  500), mindbh = 10, type = "count", include = c("A"))
{
  ptm <- proc.time()
  if (is.null(censdata2))
    censdata2 <- censdata
  n <- nrow(censdata2)
  output <- matrix(NA, ncol = 2, nrow = n)
  dimnames(output)[[2]] <- c("con.sp", "het.sp")
  if (type == "count") {
    spd <- subset(censdata, !is.na(gx) & !is.na(gy) & !duplicated(tag) &
      dbh >= mindbh & status %in% include)
    for (i in 1:n) {
      focal <- censdata2[i, ]
      if (is.na(focal$gx) | is.na(focal$gy) | duplicated(focal$tag))
        output[i, 1:2] <- NA
      else {
        poly <- with(focal, spoints(c(gx - r, gy - r,
          gx - r, gy + r, gx + r, gy + r, gx + r, gy -
            r)))
        use <- inpip(spoints(rbind(spd$gx, spd$gy)),
          poly)
        if (length(use) == 0)
          output[i, 1:2] <- 0
        else {
          incircle <- sqrt(dsquare(spoints(rbind(spd$gx[use],
```

```

        spd$gy[use])), spoints(rbind(focal$gx, focal$gy)))) <=
        r
    nn <- use[incircle]
    consp <- spd$sp[nn] == focal$sp
    area <- CalcRingArea(data.frame(gx = focal$gx,
        gy = focal$gy), r, plotdim)$each
    output[i, 1] <- length(nn[consp]) * (pi * r^2/area)
    output[i, 2] <- length(nn[!consp]) * (pi *
        r^2/area)
    }
}
if (i %in% seq(5000, n + 5000, 5000))
    cat(i, "of", n, " elapsed time = ", (proc.time() -
        ptm)[3], "seconds", "\n")
}
}
if (type == "basal") {
    spd <- subset(censdata, !is.na(gx) & !is.na(gy) & !is.na(dbh) &
        dbh >= mindbh & status %in% include)
    for (i in 1:n) {
        focal <- censdata2[i, ]
        if (is.na(focal$gx) | is.na(focal$gy))
            output[i, 1:2] <- NA
        else {
            poly <- with(focal, spoints(c(gx - r, gy - r,
                gx - r, gy + r, gx + r, gy + r, gx + r, gy -
                r)))
            use <- inpip(spoints(rbind(spd$gx, spd$gy)),
                poly)
            if (length(use) == 0)
                output[i, 1:2] <- 0
            else {
                incircle <- sqrt(dsquare(spoints(rbind(spd$gx[use],
                    spd$gy[use])), spoints(rbind(focal$gx, focal$gy)))) <=
                r
                nn <- use[incircle]
                area <- CalcRingArea(data.frame(gx = focal$gx,
                    gy = focal$gy), r, plotdim)$each
                BA <- circlearea(spd$dbh[nn]/20)
                consp <- spd$sp[nn] == focal$sp
                output[i, 1] <- sum(BA[consp], na.rm = TRUE) *
                    (pi * r^2/area)
                output[i, 2] <- sum(BA[!consp], na.rm = TRUE) *

```

```

        (pi * r^2/area)
      }
    }
    if (i %in% seq(5000, n + 5000, 5000))
      cat(i, "of", n, " elapsed time = ", (proc.time() -
        ptm)[3], "seconds", "\n")
  }
}
cat("Total elapsed time = ", (proc.time() - ptm)[3], "seconds",
  "\n")
return(output)
}

```

RipUvK

```

function (splitdata, plotdim = c(1000, 500), rseq = c(5, 10,
  20, 30, 40, 50), mindbh = 10, xcol = "gx", ycol = "gy", debug = FALSE,
  show = FALSE)
{
  spp.names <- names(splitdata)
  plotarea <- plotdim[1] * plotdim[2]/10000
  omega <- matrix(nrow = length(spp.names), ncol = length(rseq))
  rip.list <- Ovalue <- list()
  abund <- numeric()
  poly <- spoints(c(0, 0, plotdim[1], 0, plotdim[1], plotdim[2],
    0, plotdim[2]))
  sppcounter <- 1
  for (i in 1:length(spp.names)) {
    if (mindbh == 0)
      size <- rep(0, dim(splitdata[[i]])[1])
    else size <- splitdata[[i]]$dbh
    if (is.null(splitdata[[i]]$status))
      alivestatus <- rep("A", dim(splitdata[[i]])[1])
    else alivestatus <- splitdata[[i]]$status
    gx <- (splitdata[[i]][, xcol])
    gy <- (splitdata[[i]][, ycol])
    include <- which(alivestatus == "A" & size >= mindbh &
      gx >= 0 & gx < plotdim[1] & gy >= 0 & gy < plotdim[2])
    inc <- (alivestatus == "A" & size >= mindbh & gx >= 0 &
      gx < plotdim[1] & gy >= 0 & gy < plotdim[2])
    gx <- gx[include]
    gy <- gy[include]
  }
}

```



```

abund[i] <- length(gx)
if (debug)
  browser()
if (abund[i] > 1) {
  sppi.pts <- spoints(rbind(gx, gy), length(gx))
  rip.list[[i]] <- khat(as.points(sppi.pts), poly = poly,
    rseq, newstyle = TRUE)
  ringarea <- Annuli(data.frame(gx, gy), rseq, plotdim)
  Ovalue[[i]] <- colSums(rip.list[[i]]$counts)/(ringarea$total/10000)
  rip.list[[i]]$area <- ringarea$each
  lambda <- abund[i]/plotarea
  if (debug)
    browser()
  omega[i, ] <- Ovalue[[i]]/lambda
  if (show) {
    if (sppcounter == 1)
      plot(i, omega[i, 1], log = "y", xlim = c(0,
        1200), ylim = c(0.1, 60))
    else points(i, omega[i, 1])
  }
  sppcounter <- sppcounter + 1
}
else rip.list[[i]] <- Ovalue[[i]] <- NA
}
names(Ovalue) <- names(rip.list) <- names(abund) <- rownames(omega) <- spp.names
nopts <- length(rseq)
midpts <- c(0, rseq[-nopts]) + diff(c(0, rseq))/2
return(list(K = rip.list, O = Ovalue, omega = data.frame(omega),
  abund = abund, midpts = midpts))
}

```

Overview

Some functions work on areas, other functions work on counts.

The function `circlearea` is trivial; while the `partialcirclearea` makes edge corrections considering the dimensions of the plot. That will be important in forest applications. It seems the most interesting piece of functionality, apparently Condit implemented Goreaud & Pélissier (1999, *Journal of Vegetation Science* 10: 433-438), which are smart analytical solutions.

`CalcRingArea` seems to be just the accumulated sum of (possibly truncated) circles, one for each point in a dataset. Note:

- it calculates areas of circles, not rings, despite its name
- it does not correct for overlap, so it counts many times the same areas

The area of rings (“annuli”) is calculated by the `Annuli` function, which repeatedly calls `CalRingArea` and subtracts small circles from larger circles successively.

`NDcount` and `NeighborDensities` try to do similar things. `NDcount` (author Tania Brenes) wraps `splan::khat` function to count neighbors with edge correction. `NeighborDensities` (author Rick Condit) incorporates filters, the possibility of counting neighbors from a different dataset (e.g. adult trees neighboring seedlings), and “counting” as accumulated basal area instead of individual counts. It depends on specific column names like “status”. There are risky unit conversions inside, so it does make strong assumptions about the units of the input: clearly a vulnerable point.

`RipUvK` is the highest-level function in this group of functions. It runs over each species calculating a series of spatial statistics for con-specifics at different distances. It seems easy to use, it is not very long. It is not very flexible, and calls external functions of undefined packages, probably `splan` package: `khat`, `spoints`.

Because the code has obvious points of vulnerability, I decide to rewrite. I do not run formal tests.

Design principles for alternative code

Ideally, replacement code should incorporate the flexibility to count neighbors of any class to any class, for example:

- adult tree neighboring seedlings
- individuals of species A neighboring species A (conspecifics)
- individuals of species A neighboring species B (some heterospecifics)
- etc.

The conspecific vs. heterospecific counting, which is the most frequent, is just a special case.

Calculating distances to focal points could be relatively easy, but memory and computation is important for large datasets. Counting and calculating areas has to go hand in hand, because of edge effects. In other words: the output should be typically expressed as densities, on a per area basis, as originally.

It should be general and count everything inputted. If the user wants to focus on trees that are alive, for example, it should subset the dataset first. It does not seem robust to filter from within the function and make assumptions about column names, etc. Filter outside gives all the flexibility to the user, even if they keep lianas and trees together, or whatever they want. This is aligned with the goal of reducing dependency from “orthodox CTFS format”.

It should be unit agnostic: return everything on the same units as the input.

Ideally, the code should be fast. Solutions without loops would be preferred. Memory could be a constrain so tutorials should teach how to do things by chunks.

Edge effects are important. Ideally, we should be able to define edges of any arbitrary shape. For example, so Ngel Nyaki plot can truncate circles to the border of the forested part of the plot, not just the rectangular borders. Or do counts + areas for specific habitat types within the rectangular plot, etc.

The Ripley's K function is an statistic that measures the deviation of a given count in a given area relative to the null expectation of random placement (Poisson point process or PPP). If the counting code is fast, it could be entirely replaced by a numerical null model, which would have some advantages:

- Importantly: it would not be limited to individual counts as now, but could incorporate observations vs. expectations for other metrics, such as accumulated basal area, total biomass, etc.
- The user could model, potentially, other null expectations, not just PPP.

It seems better to develop counting+area first, then check whether a numerical comparison with null expectations is computationally feasible.

Alternative code to count

The idea is:

(1) a function to calculate, fast, distances up to certain threshold. This is useful for the distances themselves and to identify neighbors.

(2) a function to split a set of points, given a boundary and a distance, into “out”, “edge”, and “core”. This is useful to estimate error in numerical estimates focusing on “core” points, and to restrict (approximate) numerical estimates to the areas near the edge only.

(3) a smart system to define grid of certain resolution that can help calculate the areas associated to any shape. This would solve *any* edge effect of any shape, not just in rectangular windows. It is numerical, not analytic, so computation and memory matter, so tolerances and resolutions matter. We can use “core” points and circles to estimate the error associated to a grid of a given resolution.

(4) the function that calculates distances and identify neighbors is useful to count them or summarize their abundance in different ways: accumulated biomass, or basal area, etc. That can be separated, and build on the who-is-who object describing neighborhoods.

(5) a null model can substitute Ripley's K test, a generalization for other abundance metrics, not just counts. If the null model is constrained to a given boundary, it would reflect edges, so deviations from the model would solve for edge effects. The simplest null model is the Poisson point pattern (= uniformly random locations, independently for each point), as in Ripley's K.

But there are other models that could be applied, including the constrained torus translation for irregular boundaries.

(6) calculations for rings are the same as for circles and then applying sequential subtraction. The function can output both, so we do not have to develop code to go from one to the other or *vice versa*.

First, a function to calculate, fast, distances up to certain threshold. This is useful for the distances themselves and to identify neighbors.

```
# Fast calculation of distances up to a given radius:
fast_distances_up_to_r <- function(query, reference, radius, approx = 0) {
  query <- as.matrix(query)
  reference <- as.matrix(reference)

  storage.mode(query) <- "double"
  storage.mode(reference) <- "double"

  nn <- dbscan::frNN(
    x = reference,
    query = query,
    eps = radius,
    sort = FALSE,
    approx = approx
  )

  k <- lengths(nn$id)

  data.table::data.table(
    query_i = rep(seq_along(k), k),
    reference_j = unlist(nn$id, use.names = FALSE),
    dist = unlist(nn$dist, use.names = FALSE)
  )[dist <= radius]
}

# Check

n = 100000 # 100-300 k points
radius = 50 # something realistic, rarely > 100
p_focal = 0.10 # proportion of the forest in the focal class

xy <- cbind(x = runif(n, 0, 500), y = runif(n, 0, 500))
focal_xy <- xy[1:round(p_focal*n),]
```

```
reference_xy <- xy
```

```
system.time(d <- fast_distances_up_to_r(focal_xy, reference_xy, radius = radius, approx = 0))
```

```
      user  system elapsed  
4.137    0.823    4.970
```

```
dim(d)
```

```
[1] 28836426      3
```

```
# Surprisingly fast, but the output is really big (obviously). It returns 1M rows per second
```

Now, an auxiliary function to split a given set of points into “out”, “edge” and “core”, given a boundary and a distance threshold. This has a couple of useful applications in this workflow. It takes advantage of the fast calculation of distances (function above).

```
core_vs_edge <- function(xy, boundary, radius,  
                          xy_cols = c("x", "y"),  
                          boundary_step = radius / 10,  
                          assume_inside = FALSE,  
                          approx = 0) {  
  
  xy <- as.data.frame(xy)  
  query_xy <- as.matrix(xy[, xy_cols, drop = FALSE])  
  storage.mode(query_xy) <- "double"  
  
  W <- sf::st_geometry(boundary)  
  W <- sf::st_make_valid(W)  
  W <- sf::st_union(W)  
  
  # Optional but safer unless you already know all points are inside.  
  if (assume_inside) {  
    inside <- rep(TRUE, nrow(query_xy))  
  } else {  
    pts <- sf::st_as_sf(  
      xy,  
      coords = xy_cols,  
      crs = sf::st_crs(W),  
      remove = FALSE
```

```

    )
    inside <- lengths(sf::st_intersects(pts, W)) > 0
  }

  # Densify boundary once, then use fast_distances_up_to_r()
  B <- sf::st_boundary(W)
  B <- sf::st_segmentize(B, dfMaxLength = boundary_step)

  boundary_xy <- sf::st_coordinates(B)[, c("X", "Y"), drop = FALSE]
  boundary_xy <- unique(boundary_xy)
  storage.mode(boundary_xy) <- "double"

  # Safety margin: nearest sampled boundary point may be offset along the boundary.
  search_radius <- radius + boundary_step / 2

  near <- rep(FALSE, nrow(query_xy))

  ii <- which(inside)
  if (length(ii)) {
    d <- fast_distances_up_to_r(
      query = query_xy[ii, , drop = FALSE],
      reference = boundary_xy,
      radius = search_radius,
      approx = approx
    )

    near[ii[unique(d$query_i)]] <- TRUE
  }

  # Build output into three classes:
  location <- rep(NA, length(inside))
  location[!inside] <- "out"
  location[inside & near] <- "edge"
  location[inside & !near] <- "core"
  location
}

# Example with a complex boundary:
library(sf)

```

Warning: package 'sf' was built under R version 4.5.2

Linking to GEOS 3.13.0, GDAL 3.8.5, PROJ 9.5.1; sf_use_s2() is TRUE

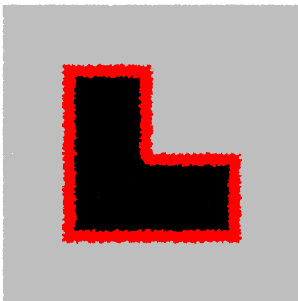
```
boundary <- sf::st_sfc(
  sf::st_polygon(list(rbind(
    c(100, 100),
    c(100, 400),
    c(250, 400),
    c(250, 250),
    c(400, 250),
    c(400, 100),
    c(100, 100)
  )))
)

xy <- data.frame(xy)

loc <- core_vs_edge(
  xy = xy,
  boundary = boundary,
  radius = 20,
  boundary_step = 0.1
)

COL <- rep("gray", ncol(xy))
COL[loc == "out"] <- "gray"
COL[loc == "edge"] <- "red"
COL[loc == "core"] <- "black"

plot(boundary, col = NA, border = "black", xlim = c(0, 500), ylim = c(0, 500))
points(xy$x, xy$y, cex = 0.1, col = COL)
```



Now we look for smart ways of creating dense (but not excessively dense) grids that can be good descriptions of the shapes at the edges: intersections of circles and (possibly complex)

boundaries. It is better to separate this, because in practice the user will be running loops inside loops, and we want to keep these calculations outside the costly loops. We have to prepare tutorials for these things.

```
# The grid cannot go beyond the boundaries, so the bounding box of the (possibly complex) boundary
make_grid_inside_boundary <- function(boundary, step = 1)
{
  bb <- sf::st_bbox(boundary)

  grid <- expand.grid(x = seq(from = bb["xmin"] - step, to = bb["xmax"] + step, by = step),
                    y = seq(from = bb["ymin"] - step, to = bb["ymax"] + step, by = step))

  W <- sf::st_geometry(boundary)
  W <- sf::st_make_valid(W)
  W <- sf::st_union(W)
  pts <- sf::st_as_sf(grid, coords = c("x", "y"), remove = FALSE)
  inside <- lengths(sf::st_intersects(pts, W)) > 0
  grid <- grid[inside,, drop = FALSE]
  grid
}

# The valid resolution of the grid is not solvable with a given formula,
# as it is a form of circle + grid intersection, difficult geometrical problem.
# But it can be approached numerically, in a relatively easy way, using
# the empirical data. The idea is: calculate areas of circles for "core"
# points (that is, not affected by edge effects), and compare that with
# the known area of circles of a given radius. Once that is known,
# use a heuristic for a smaller step: that should be a conservative estimate
# that will constrain error in the corners in pseudo-worst-case scenarios.
# The user defines a tolerance: the step size derives from that,
# the radius of the circles, and the distribution of the empirical points.

guess_sufficient_grid_resolution <- function(xy, radius, boundary, tol = 0.01)
{
  # tol: the accepted error in area

  # To measure error empirically, test with a subset of
  # the query that is not affected by edge effects:

  # Use a coarse step to initialize
  bb <- sf::st_bbox(boundary)
```



```

step = min(c((bb["ymax"] - bb["ymin"]) / 3,
             (bb["xmax"] - bb["xmin"]) / 3))

# Iterate until we reach the desired tolerance:
error = 999
while(error > tol)
{
  step = step / 2 # smaller step
  grid <- make_grid_inside_boundary(boundary = boundary, step = step)

  # Do the estimate using the core points only:
  core <- which(core_vs_edge(xy = xy, boundary = boundary, radius = radius) == "core")
  d.test <- fast_distances_up_to_r(xy[core,], grid, radius, approx = 0)
  A.estimated <- tabulate(d.test$query_i, nbins = length(core)) * step^2
  A.true = pi*radius^2
  error = mean(abs(as.numeric(A.estimated) - A.true) / A.true)
  #print(error)
}

# As a general rule, error will be larger near the boundary,
# and greater if the boundary is very complicated.
# So, to apply this, we need greater resolution (smaller steps).
# How smaller? Well... there is not a neat answer for that.
# But ChatGPT says that this is a useful conservative heuristic,
# derived from Steiner / parallel-body argument:
K = 1 + 4 / pi
suggested_step = step / K
ppc = floor(pi*(radius^2)/(suggested_step^2)) # points per circle

# There are other rules of thumb and analytical results for other cases:
# To constrain the maximum possible error:
conservative_step = tol * radius / (2 * sqrt(2))
# Less conservative: to constrain average error
less_conservative_step = radius * (2 * tol)^(2/3)

# The output with everything of value:
c(suggested_step = suggested_step,
   conservative_step = conservative_step,
   less_conservative_step = less_conservative_step,
   radius = radius, tol = tol, error = error, points_per_circle = ppc)
}

```

To put things together, we have to combine (1) the standard formula for the circle area for the core focal points, and (2) the grid-derived estimates near the edge. For this second point, we cross: (a) the subset of focal points closer than R to the boundary, and (b) the subset of grid points closer than $2R$ to the boundary, being R the radius of the scale being evaluated. All this will make more sense within loops that do the entire calculation at different scales.

```
errors.etc <- guess_sufficient_grid_resolution(xy = focal_xy, boundary = boundary, radius = radius)

step = errors.etc["suggested_step"]

grid <- make_grid_inside_boundary(boundary, step = step)
dim(grid) # can be large
```

```
[1] 9075    2
```

```
ceo_focal <- core_vs_edge(xy = focal_xy, boundary = boundary, radius = radius)
ceo_grid  <- core_vs_edge(xy = grid, boundary = boundary, radius = 2*radius) # out of loops!
table(ceo_focal)
```

```
ceo_focal
core edge out
 672 2184 7144
```

```
table(ceo_grid)
```

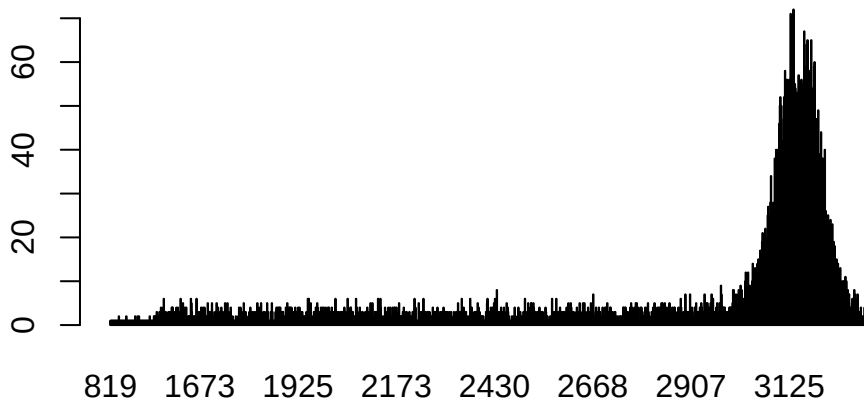
```
ceo_grid
edge
9075
```

```
focal_xy_edge <- focal_xy[ceo_focal == "edge",,drop = FALSE]
grid_for_edge <- grid[ceo_grid == "edge",,drop = FALSE]

# Now take advantage of the neighbor-identifying workhorse:
d <- fast_distances_up_to_r(focal_xy_edge, grid_for_edge, radius, approx = 0)

# Put all the areas together: initialize with full circles,
# then overwrite with the areas calculated for the edge cases:
A <- rep(pi*radius^2, nrow(focal_xy))
A[ceo_focal == "edge"] <- tabulate(d$query_i, nbins = nrow(focal_xy_edge)) * step^2
```

```
# Count the number of neighbors and express that as trees / area:
d <- fast_distances_up_to_r(focal_xy, reference_xy, radius, approx = 0)
number_of_neighbors <- tabulate(d$query_i, nbins = nrow(focal_xy))
barplot(table(number_of_neighbors)) # if random locations: Poisson ~ normal distribution
```

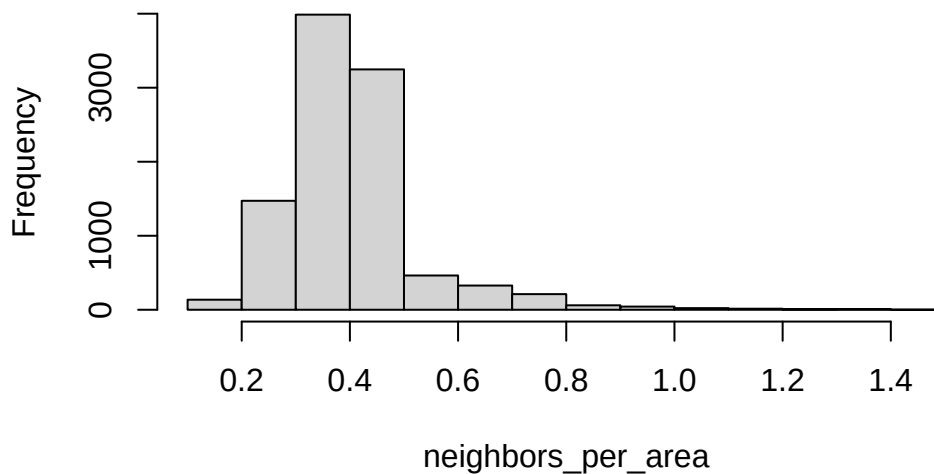


```
# Express as densities, edges corrected:
length(number_of_neighbors) == length(A)
```

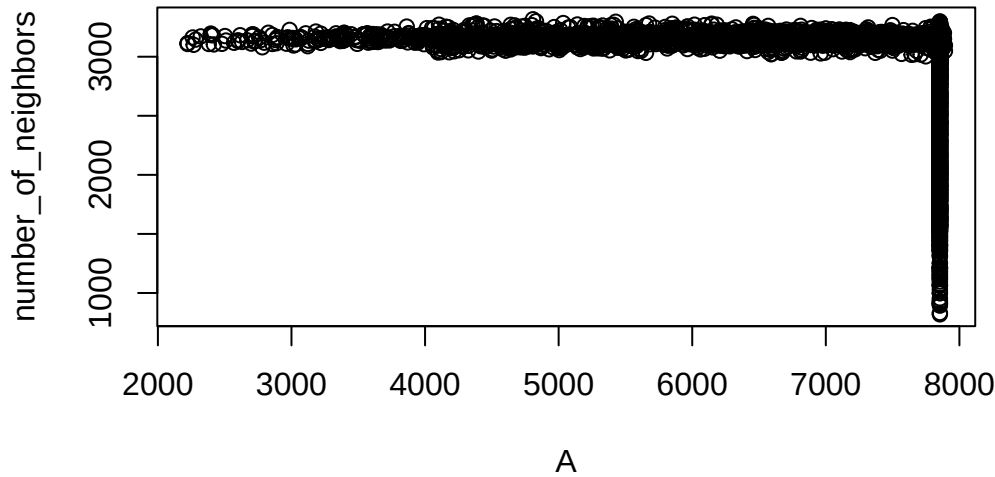
```
[1] TRUE
```

```
neighbors_per_area <- number_of_neighbors / A
hist(neighbors_per_area)
```

Histogram of neighbors_per_area



```
plot(A, number_of_neighbors)
```



If the calculations are fast enough, we could also run null models. Null models have advantages:

- The simplest null model is uniform random distribution (PPP), and this will end being very similar to Ripley's K.
- The PPP null model can be very naïve for some ecological analyses, so the user could incorporate different null models, like the torus translation or constrained torus translation.
- The metric of interest in a null model approach could be any, not just counts of neighbors as implicit in Ripley's K. It could be biomass, sum of basal area, etc. Condit incorporated some of this in the description of neighborhoods but that did not fit into the Ripley's K analyses.
- The null models already incorporate the expectation of fewer neighbors for focal points near the edges, so they can be a direct way of correcting for edge effects for boundaries or arbitrary shape.

In general, null models have nice properties for generalization and flexibility. Their main practical limitations are: (1) the tremendous responsibility on the definition of a reasonable / useful expectation, and (2) computation and storage.